EX02 Report: Shared State

Student Name: David Mairhofer **Student ID:** csbc2382

Setup

Two `t2.micro` Amazon Linux 2023 instances with terraform (main.tf, taken from last week's exercise). It needs the learner lab credentials in the environment:

```
export AWS_ACCESS_KEY_ID=... AWS_SECRET_ACCESS_KEY=... AWS_SESSION_TOKEN=...
terraform init
terraform apply
# public_ips = ["54.196.53.167", "54.198.239.167"]
```

You'll need to change the ingress rule `cidr_blocks` in main.tf to your IP or 0.0.0.0/0 to allow all IPs. All commands run from this folder, `labsuser.pem` is from the learner lab:

```
ssh -i labsuser.pem ec2-user@<ip>
scp -i labsuser.pem counter.py store.py incr.py ec2-user@<ip>:~/
```

A = `54.196.53.167`, B = `54.198.239.167`.

Task 1 - Replicated counter

CLARIFICATION: I used deepseek to help me write the python version. Look into `cpp/README.md` for how the C++ version that I made compares.

counter.py

```
python3 counter.py <A|B> <my_port> <peer_host> <peer_port> <increments> <basic|improved>
```

- `<peer_host>`:I use the private IP of the other node (which works because they are in the same VPC group); or just a dummy value for `A` (as it starts a server)
- `<peer_port>`: The port of the peer node (dummy value for `A` as it starts a server)
- `<increments>`: The number of increments to do
- `<basic|improved>`: `basic` (default, task 1.1-1.3) or `improved` (task 1.4; default if you pass anything else as this argument or omit it) "protocol" for the counter
- `A` listens, `B` connects

Every increment is sent to the other node right away. Both have a separate receiver thread (and lock for synchronization with the main 'incrementing' thread) that merges the incoming state with the local one with `max()`. There is no sync phase at the end (which I had in the beginning, but under the assumption that you don't loose a message, that becomes useless).

```
state = [0, 0]
def receiver():                         # thread, runs the whole time
    for line in f:
        state[0] = max(state[0], peer[0])   # merge
        state[1] = max(state[1], peer[1])   # for task 1.4
...
for _ in range(increments):
    if mode == "basic":          # task 1.1-1.3
        state[0] += 1
    else:
        state[me] += 1
    conn.sendall(f"{state[0]} {state[1]}\n".encode())   # one message per increment
```

1.1 - Basic version

For a simple test, `A` increments 100 times, `B` none. `B` gets one message per A increment. Because of TCP guerantees both end at 100:

```
python3 counter.py A 5002 127.0.0.1 5002 100 basic   # on A
python3 counter.py B 5002 172.31.87.249 5002 0 basic # on B
```

```
A: local 0.1775 ms, send 0.890 ms, sync 0.063 ms, A=100 B=0 total=100
B: local 0.0000 ms, send 0.000 ms, sync 2.008 ms, A=100 B=0 total=100
```

1.2 - Concurrent increments

One increment on each node:

```
python3 counter.py A 5002 127.0.0.1 5002 1 basic   # on A
python3 counter.py B 5002 172.31.87.249 5002 1 basic # on B
```

```
A: local 0.0023 ms, send 0.025 ms, sync 0.061 ms, A=1 B=0 total=1
B: local 0.0018 ms, send 0.038 ms, sync 1.238 ms, A=1 B=0 total=1
```

Result is 1 but should be 2. Both increment from 0 to 1, then `max(1, 1)` keeps one of the two increments, so the nodes agree but the value is wrong. (I'm kind of cheating, as I currently force both participants to increment once before they start receiving messages and merging to show that this can happen if the order of operations is right)

It gets worse with more increments: with `n` increments on each node the agreed value always lands between `n` and `2n`. Each node keeps at least its own `n` increments, and no node can hold more than the `2n` increments that happened in total. 100 increments each (correct: 200) gave 138, 158, 160, 170, 171 and 198 in six local runs. With n=30 it gave: 48, 59, 56, 42. The more of its own increments a node does after the peer's counter got merged, the closer the value gets to `2n`; if both count evenly, more increments overwrite each other and it stays near `n`. Being equal says nothing about being right; idk how I would debug something like this if I were to run into this problem in a real system.

1.3 - Lost, duplicated and old messages

- Lost message: because each message carries the full counter, a lost message only matters if it was the last one/ones
- Duplicate message: `max` is idempotent, the same value twice changes nothing
- Old message after a newer one: `max` ignores the smaller value, the counter never goes backwards

1.4 - Improved version

Each node increments only its own field:

```
python3 counter.py A 5002 127.0.0.1 5002 100 improved   # on A
python3 counter.py B 5002 172.31.87.249 5002 100 improved # on B
```

```
1 increment each:    A=1 B=1 total=2
100 increments each: A=100 B=100 total=200
```

The two increments of 1.2 are no longer lost, the total is just the sum of both slots.

Task 2 - Single-copy counter

With a single copy, one process has to own the counter, so I called that the "store". The nodes only send it commands like `INCR`/`GET`/`SET` and get the answer back. I didn't want to install Redis so I wrote store.py: the counter is in memory, one connection = one command, a lock makes each command atomic

```
with lock:
    if op == "INCR":
        counter += 1
    elif op == "SET":
        counter = int(args[0])
```

```
python3 store.py 5001   # on A; executed in another (3rd) ssh session/terminal
python3 incr.py <host> <port> <increments> <safe|unsafe> [start_epoch]
```

(`safe` for task 2.1, `unsafe` for task 2.2; default is `unsafe`)

2.1 - Safe increments

Both nodes start at the same time, 100 `INCR` each:

```
echo $(( $(date +%s) + 10 ))                          # my local machine
# to get a start time so both nodes start at the same time, as they are pretty fast

# on A; connects to store on localhost
python3 incr.py 127.0.0.1 5001 100 safe <copied timestamp number>
```

```
# on B
python3 incr.py 172.31.87.249 5001 100 safe <copied timestamp number>
```

```
A: safe: 22.3 ms
B: safe: 64.1 ms
final counter: 200
```

Nothing is lost.

2.2 - Concurrent updates

```
value = int(store(host, port, "GET"))       # read
time.sleep(0.005)                           # widen the race window
store(host, port, f"SET {value + 1}")       # write old value + 1
```

```
echo $(( $(date +%s) + 10 ))
python3 incr.py 127.0.0.1 5001 100 unsafe <copied timestamp number>     # on A
python3 incr.py 172.31.87.249 5001 100 unsafe <copied timestamp number> # on B
```

```
A: unsafe: 555.5 ms
B: unsafe: 672.3 ms
final counter: 138   (instead of 200, 62 increments lost)
```

Both read the same value before the other one wrote, so they overwrote each other. `INCR` cannot do that, the whole +1 happens inside the store.

A second update does not fail and does not run in parallel: it waits on the store's lock until the first one is done.

Task 3 - Comparison

what	time
replicated: 100 local increments	0.18-0.22 ms
replicated: sending those 100 messages	0.7-1.1 ms
replicated: waiting for the peer's last message	0.06-1.0 ms
single-copy: 100 INCR from A (localhost store)	22 ms
single-copy: 100 INCR from B	64 ms
store unsafe: 100 GET/SET with 5 ms sleep	~0.6 s

A local increment costs a few µs and one replicated message about 10 µs. A store `INCR` opens a new TCP connection and waits for the reply, that is the ~0.2 ms per increment from A and ~0.6 ms from B.

Concurrent updates in the single-copy version would mean one node waits for the other to finish. But surprisingly, this isn't noticable, as the lock only surrounds the increment, which is fast; the slow part is the network round trip (which happens in parallel from the 2 nodes)

3.2

1. Where does synchronization happen in each implementation?

Replicated: state is and synchonization heppens in the nodes.
Single-copy: state is and synchonization heppens in the store.

2. What happens if the two nodes cannot communicate temporarily?

Replicated: both keep working with their local copy, they converge later
Single-copy: nobody can increment without the store

3. What happens if the shared data store is unavailable?

Replicated: only the sync is delayed
Single-copy: increments fail or block

4. Which approach would you choose if increments must be immediately visible to both nodes? Why?

Single-copy, both nodes always read the same value. A replica only sees an increment when the message for it arrived.